

Type Erasure Requirements For Future Trivial Relocation Design

Document #: P3937R0 [[Latest](#)] [[Status](#)]
Date: 2025-12-15
Project: Programming Language C++
Audience: EWG
Reply-to: Mingxin Wang
<mingxwa@outlook.com>
Zhihao Yuan
<zy@miator.net>

Contents

- [1 Introduction](#)
- [2 Motivation](#)
 - [2.1 Simplicity](#)
 - [2.2 Performance](#)
 - [2.2.1 Fast Path in Proxy](#)
 - [2.2.2 The Cost of Conflation](#)
 - [2.2.3 Benchmark Evidence](#)
 - [2.3 Security](#)
 - [2.3.1 Type Erasure vs. Virtual Functions](#)
 - [2.3.2 Safety of Fixup APIs](#)
- [3 Requirements For A Future Design](#)
 - [3.1 Definition: “Trivially Relocatable” Means Bitwise](#)
 - [3.2 Non-Movable Types Can Be Trivially Relocatable](#)
 - [3.3 Conditional Trivial Relocation](#)
 - [3.4 Impact On Proxy](#)
 - [3.5 Lifetime Semantics](#)
 - [3.6 Guidance \(Low Level Usage\)](#)
- [4 Interaction With Existing Papers](#)
 - [4.1 P2786 \(Trivial Relocatability For C++26\)](#)
 - [4.2 P3780 \(Detecting Bitwise Trivially Relocatable Types\)](#)
 - [4.3 P3858 \(restart_lifetime / start_lifetime_at\)](#)

- [5 Summary](#)
 - [5.1 Next Steps \(non-normative\)](#)
- [6 References](#)

§ 1 Introduction

In the C++26 cycle, [\[P2786R13\]](#) was proposed to introduce trivial relocation to the C++ standard. However, National Body comments (specifically [US 44-082](#)) during the ballot period raised significant concerns about the definition of “trivial”, leading to consensus to remove the feature from C++26 and to revisit the design.

This paper outlines the essential requirements for any future trivial relocation design from the perspective of high-performance type erasure libraries, such as the proposed `std::proxy` ([\[P3086R5\]](#)). We argue that **bitwise relocation must be the basis of trivial relocation**, rather than a special case or optimization. This fundamental definition is necessary to achieve simplicity, performance, and security.

§ 2 Motivation

We propose that “trivially relocatable” be strictly defined as “bitwise relocatable” (`memcpy` and `memmove`).

§ 2.1 Simplicity

Defining trivial relocation strictly as bitwise operations aligns with the principle behind **Regular type** [\[DeSt98\]](#): keeping assumptions minimal to maximize flexibility. By defining relocation as the transfer of object representation, we decouple it from the semantic concept of “movability”. A type need not be move-constructible to be trivially relocatable. This allows classes to maintain stricter invariants by avoiding the “moved-from” states (e.g., a smart pointer that is never null), simplifying class design.

For example, the `allocated_ptr` class in Proxy (see [include/proxy/v4/proxy.h](#)) manages a heap-allocated object. It is designed to be immutable and never null, so it deletes its move constructor:

```
template <class T, class Alloc>
class allocated_ptr {
    T* ptr_;           // exposition only
    Alloc alloc_;      // exposition only, assuming trivially copyable

public:
    template <class... Args>
    explicit allocated_ptr(const Alloc& alloc, Args&&... args)
        : ptr_(allocate(alloc, std::forward<Args>(args)...)), alloc_(alloc) {}

    allocated_ptr(allocated_ptr&& rhs) = delete; // No move constructor
    ~allocated_ptr() { deallocate(alloc_, ptr_); }
};
```

Despite being non-movable, `allocated_ptr` is safe to relocate via `memcpy` because its internal representation (a pointer and an allocator) is position-independent. A strict bitwise definition of trivial relocation allows such types to participate in optimized relocation paths without forcing them to compromise their design by exposing a move constructor and a corresponding “moved-from” state.

§ 2.2 Performance

Relocation, in type-erased contexts, falls into two operational classes:

- **Bitwise Relocation:** Copy `sizeof(T)` bytes to the destination and end the source lifetime. This path typically compiles to `memcpy` or inline loads/stores, and no user code runs.
- **Move + Destroy:** Invoke the move constructor at the destination and the destructor at the source. This path executes arbitrary user code.

Types that require representation fixups (e.g., pointer authentication adjustments) but not a full move still require executing type-specific code. From erased contexts, this shares the key characteristic of Move + Destroy: it is **type-aware relocation** and therefore not eligible for the uniform fast path.

§ 2.2.1 Fast Path in Proxy

proxy stores vtable-like metadata and a byte-array buffer that holds a handle to the implementation object (a raw pointer, `std::unique_ptr`, an offset/arena handle, or a small in-place owning handle) [P3086R5]. It is technically feasible to implement such handles as bitwise relocatable, and proxy does so in practice. Consequently, proxy defaults a facade’s relocatability to trivial (bitwise), enabling the compiler to lower relocation into inline loads/stores across all admitted types.

Consider the implementation in proxy (simplified from `include/proxy/v4/proxy.h`):

```
// Dispatcher for non-trivial relocation (requires function call)
struct relocate_dispatch : internal_dispatch {
    template <class T, class F>
    static void operator()(T&& self, proxy<F>& rhs) {
        std::construct_at(std::addressof(rhs), std::forward<T>(self)); // Calls
                                move ctor
    }
};

template <facade F>
class proxy {
public:
    proxy(proxy&& rhs) noexcept(F::relocatability >= constraint_level::nothrow) {
        if (rhs.meta_.has_value()) {
            if constexpr (F::relocatability == constraint_level::trivial) {
                // FAST PATH: Direct memcpy, no dispatch
                memcpy(ptr_, rhs.ptr_, sizeof(ptr_));
                meta_ = rhs.meta_;
                rhs.meta_.reset();
            } else {
                // SLOW PATH: Indirect call to relocate_dispatch
                proxy_invoke<relocate_dispatch,
```

```

        void(proxy&) && noexcept(F::relocatability == constrain-
t_level::nothrow)>{
            std::move(rhs), *this);
        }
    } else {
        meta_.reset();
    }
}

private:
    details::meta_ptr<typename details::facade_traits<F>::meta> meta_;
    alignas(F::max_align) std::byte ptr_[F::max_size];
};

```

Because proxy stores its witness handle inline next to facade metadata, every move of the wrapper is literally a relocation of that byte buffer. Ordinary `std::vector` reallocation, coroutine frame materialization, arena compaction, or capturing proxies into higher-order utilities all run through this path. Keeping relocation on a memcpy-only fast path is what lets proxy compete with traditional virtual-based polymorphism without paying per-move dispatch costs.

Thus, for type erasure, any relocation requiring code execution (fixups) incurs the same dispatch overhead as a move operation. Erased optimizations rely on a uniform contract (e.g., `F::relocatability` in proxy) across all admitted types, not per-concrete type properties. If “trivial relocation” permits fixups, a type-erasure wrapper cannot guarantee memcpy is safe for the whole admitted set. This forces dispatch on every relocation, putting proxy or any other modern type-erasure at a performance disadvantage.

§ 2.2.2 The Cost of Conflation

If the standard definition of “trivial relocation” includes types requiring fixups (as [\[P2786R13\]](#) did), type-erasure wrappers are forced to be conservative. They must emit indirect calls for all types they admit under that contract. This prevents compilers from lowering relocation of type-erasure wrappers into inline loads/stores; instead, it forces control-transfer (calls/jumps).

§ 2.2.3 Benchmark Evidence

Standard polymorphic small-buffer wrappers today (e.g., `std::function`, `std::move_only_function`, `std::any`) generally perform moves via an indirect path that executes user move construction and destruction logic even when a raw byte relocation would suffice. Proxy, as an example library, emits a direct byte copy only when a facade’s relocatability level is `trivial`; otherwise, an indirect dispatch is required.

The following benchmark demonstrates the **cost of this indirection**. The “nothrow” path simulates the baseline overhead of a relocation mechanism that cannot assume bitwise semantics (and thus requires a function call), compared to the “trivial” path, which emits a direct memcpy. Each data point comes from the nightly Proxy CI run (GitHub Actions standard runners) and reappears consistently across runs.

	Operating System	Kernel Version	Architecture	Compiler
MSVC on Windows	Microsoft Windows Server 2025 Datacenter	10.0.26100	AMD64	Microsoft (R) C/C++ Optimizing Compiler Version 19.44.35221 for x64
GCC on Ubuntu	Debian GNU/Linux 13 (trixie)	6.11.0-1018-azure	x86_64	g++ (GCC) 15.2.0
Clang on Ubuntu	Ubuntu 24.04.3 LTS	6.11.0-1018-azure	x86_64	Ubuntu clang version 21.1.8 (+20251202083326+f68f64eb8130-1_exp120251202083450.66)
Apple Clang on macOS	macOS 15.7.2	24.6.0	arm64	Apple clang version 17.0.0 (clang-1700.0.13.5)
NVIDIA HPC on Ubuntu	Ubuntu 24.04.3 LTS	6.11.0-1018-azure	x86_64	nvc++ 25.11-0 64-bit target on x86-64 Linux -tp znver3
Intel oneAPI on Ubuntu	Ubuntu 24.04.3 LTS	6.11.0-1018-azure	x86_64	Intel(R) oneAPI DPC++/C++ Compiler 2025.3.1 (2025.3.1.20251023)

The benchmark (Google Benchmark style, full implementation is [open sourced on GitHub](#)) measures relocation of 1,000,000 objects (100 distinct types) for small (1 pointer) and large (6 pointers) objects:

```
PRO_DEF_MEM_DISPATCH(MemFun, Fun);

struct InvocationTestFacade : pro::facade_builder
    ::add_convention<MemFun, int() const>
    ::add_skill<pro::skills::as_view>
    ::add_skill<pro::skills::slim>
    ::build {};

struct NothrowRelocatableInvocationTestFacade : InvocationTestFacade {
    static constexpr auto relocatability = pro::constraint_level::nothrow;
};

// ... data generation helpers omitted for brevity ...

void BM_SmallObjectRelocationViaProxy(benchmark::State& state) { /* move loop */ }
void BM_SmallObjectRelocationViaProxy_NothrowRelocatable(benchmark::State& state) { /* move loop */ }
void BM_SmallObjectRelocationViaUniquePtr(benchmark::State& state) { /* move loop */ }
void BM_SmallObjectRelocationViaAny(benchmark::State& state) { /* move loop */ }
// Large object variants analogous.
```

Results (percentage speed difference, reported as $(\text{lhs} - \text{rhs}) / \text{rhs} * 100\%$, so positive values mean the left-hand side is faster):

Case	MSVC Win	GCC Ubuntu	Clang Ubuntu	Apple Clang macOS	NVIDIA HPC Ubuntu	Intel oneAPI Ubuntu
Small: proxy trivial vs proxy nothrow	+438.1%	+594.2%	+217.0%	+31.3%	+506.0%	+207.4%
Small: proxy trivial vs <code>std::unique_ptr</code>	-44.6%	-55.9%	-77.2%	-74.8%	-40.2%	-77.4%
Small: proxy trivial vs <code>std::any</code>	+371.5%	+633.3%	+823.0%	+205.8%	+81.4%	+508.4%
Large: proxy trivial vs proxy nothrow	+434.0%	+564.7%	+242.3%	+41.8%	+510.8%	+239.6%
Large: proxy trivial vs <code>std::unique_ptr</code>	-45.1%	-59.2%	-77.3%	-74.8%	-40.1%	-77.1%
Large: proxy trivial vs <code>std::any</code>	+362.7%	+549.9%	+739.8%	+281.9%	+415.5%	+485.1%

Interpretation:

- Cost of Indirection: Moving through an indirect path (“nothrow” vs. “trivial”) incurs an overhead of 2x to 6x compared to the direct code generation possible with pure bitwise relocation (Tier 1), highlighting the cost of the dispatch mechanism required to support relocation with potential fixups (Tier 2). Note that the benchmarked types do not actually require fixups, and the cost is purely the penalty of failing to assume bitwise semantics.
- `unique_ptr` move remains cheaper because it is a direct operation with a simple branch; erased wrappers can approach parity only when allowed to bypass indirect move dispatch.
- `std::any` small/large object relocation is substantially slower due to internal allocation and type policy overhead.

Conclusion: Standardizing “trivially relocatable” as a bitwise concept portably unlocks these already realized speedups.

§ 2.3 Security

Security considerations, particularly regarding Pointer Authentication (PAC), are often cited as a reason to support “fixup”-based relocation. However, this perspective requires nuance.

§ 2.3.1 Type Erasure vs. Virtual Functions

While language-level virtual functions (vtables) may require compiler-inserted fixups for PAC, library-based type erasure offers an alternative model. In a library like proxy, the “vtable” (facade metadata) is explicitly managed. This allows the library to control pointer authentication strategies directly, potentially achieving the same level of security with a smaller attack surface:

1. **No Pointer to Member:** proxy uses plain function pointers as opposed to C++ member function pointers. Member function pointers are powerful but difficult to secure efficiently (see [Clang Pointer Authentication](#)).
2. **VTable Isolation:** Function addresses stored in proxy’s vtable cannot be accessed across base class boundaries. When supporting versioned interfaces, vtables between the two versions are unrelated types. They are less likely to leak information that makes potential hardware vulnerabilities more exploitable.

Since the facade metadata is entirely under library control ([\[P3086R5\]](#)), proxy can implement PAC signing/resigning in the metadata layer without involving stored values. Facades may therefore continue to admit only bitwise-relocatable witnesses for the fast path, while the metadata enforces whatever authentication policy a platform demands. Supporting PAC-hardening does not require expanding “trivial relocation” to cover fixups.

Security hardening should not be the exclusive domain of the core language; library facilities can and should participate. Since language-level polymorphism has not evolved significantly over the past few decades, focusing relocation design exclusively on its needs risks optimizing for a local maximum while missing broader architectural improvements.

§ 2.3.2 Safety of Fixup APIs

Comparing “fixup”-based relocation to bitwise relocation also involves API safety. A “fixup” operation typically implies adjusting a pointer after it has been moved to a new address.

Recent discussions around [\[P3858R0\]](#) highlight the risks:

1. **Single-pointer fixup is unsafe:** An API that takes a single pointer and “fixes” it assumes preconditions about the dynamic type matching the static type. In a polymorphic context, mismatching these types is permitted at compile-time but causes Undefined Behavior (UB) at runtime if the fixup logic is incorrect.
2. **Two-pointer fixup needs study:** A safer alternative might involve a two-pointer operation (taking both source and destination addresses). This allows the fixup logic to access the original object state (if needed) or calculate position-dependent invariants based on the shift between source and destination addresses. Crucially, it avoids the ambiguity of operating solely on a destination pointer that may not yet point to a fully valid object. However, standardizing such an operation requires careful specification of aliasing rules and overlap behavior, which is not yet fully explored.

Given these complexities, baking a specific “fixup” model into the definition of “trivial relocation” is premature and potentially hazardous. A strict bitwise definition avoids these pitfalls entirely.

§ 3 Requirements For A Future Design

This section lists concrete requirements derived from the production use of Proxy and a survey of standard and vendor type-erasure implementations.

§ 3.1 Definition: “Trivially Relocatable” Means Bitwise

Requirement A: Specify that a trivially relocatable type is one whose objects can be relocated by copying their object representation as a sequence of bytes (bitwise relocation) using `std::memcpy` or `std::memmove` of `sizeof(T)` bytes into suitably aligned storage, after which the destination object is fully formed and the source object’s lifetime is ended, with no destructor invocation and no representation fixups.

NB guidance explicitly supporting this interpretation:

- [CN 6.9.1p9](#) [basic.types.general] Trivial relocatability should imply bitwise relocatability
- [US 46-085 11.2, 20.2.6](#) Make trivial relocation implementable by `memcpy`
- [US 45-081 11.1](#) [class.pre] Satisfy expectations that trivially relocatable types are `memcpy`-able
- [BDS2-034 6.8.1](#) [basic.types.general], 11.2 [class.prop], 20.2.6 [obj.lifetime] Permit trivial relocation via `memcpy`

Properties (mirroring the style used for trivially copyable types):

- Byte copy validity: Performing `std::memcpy` or `std::memmove` of `sizeof(T)` bytes from an object of type `T` to suitably aligned storage yields a valid object of type `T` whose value representation (excluding unspecified padding) is identical to the original.
- Overlap tolerance: Relocation via `std::memmove` permits overlapping source and destination regions consistent with existing library semantics.
- No representation fixups: Types requiring post-copy adjustment (e.g., pointer authentication) are not trivially relocatable.
- Implicit end of source lifetime: The act of relocation ends the source lifetime without invoking its destructor.
- Compiler-only determination: No user-provided specializations; the trait is implementation-determined from core language rules and subobject properties.

Rationale: Aligns industry expectation, explicitly guarantees both `memcpy` and `memmove` usability (matching precedent set by trivially copyable), and eliminates ambiguity present when “trivial” was allowed to include fixup-based transformations.

§ 3.2 Non-Movable Types Can Be Trivially Relocatable

Requirement B: Implicitly deleted or absent move operation must not disable trivial relocation ([US 47-084 11.2p2](#) [class.prop] Implicitly deleted move operation should not disable trivial relocation CWG3049). A class whose representation satisfies Requirement A remains trivially relocatable regardless of its declared move members.

Example: `allocated_ptr` (Proxy internal) stores an allocator and an object contiguously, omits a null state, and does not declare a move constructor. Its invariants do not depend on object address; a raw byte copy followed by ending the source lifetime produces a valid target object. This pattern should be recognized as trivially relocatable even though move operations are absent.

Rationale: Recognizing such designs prevents forced indirect move paths in type erasure wrappers and supports the “simplicity” goal by allowing types to avoid representing invalid states (like “moved-from” nulls) while still being relocatable.

§ 3.3 Conditional Trivial Relocation

Requirement C: Permit a type to declare that it is trivially relocatable only under a simple implementation-defined boolean condition (US 44-082 11.1 [class.pre] Conditional trivially relocatable types). If the condition holds and all constituent subobjects satisfy Requirement A, the type is trivially relocatable; otherwise, it is not.

Example (Proxy): A facade can restrict stored targets to those meeting trivial relocation, allowing the wrapper to set the eligibility boolean `true` when no representation fixups are active.

Preferred mechanism: reuse the [P2786R13] syntax `trivially_relocatable_if_eligible(bool)`. The boolean expresses eligibility; it is not a forced assertion. Libraries gain a portable, zero-overhead way to expose fast paths when the representation truly allows raw byte relocation.

Rationale: Conditional semantics let library authors surface optimized relocation only when safe, without introducing new traits or complex specialization rules.

§ 3.4 Impact On Proxy

The Proxy library fills the missing standard facility with a library-specific trait, `pro::is_bitwise_trivially_relocatable`. Its primary template is

```
template <class T>
struct is_bitwise_trivially_relocatable
    : std::bool_constant<std::is_trivially_move_constructible_v<T> &&
                        std::is_trivially_destructible_v<T>> {};
```

and users are encouraged to specialize it to `std::true_type` for additional witness types. Proxy relies on this hook to keep `proxy<F>` fast, but the approach is fragile: two translation units can disagree on the specialization, accidental opt-ins silently produce undefined behavior, and portability suffers because every ecosystem needs to rediscover the same property. The goal of this paper is to make that trait unnecessary by providing a blessed, compiler-owned facility that Proxy, and any other erased wrapper, can depend on.

To replace the bespoke hook, the standard trait must feature three characteristics that mirror Requirements A-C:

- **Keep the trait strictly bitwise:** `std::is_trivially_relocatable<T>` must report the same bitwise semantics described in Requirement A and must not allow user specialization.

That single, compiler-owned answer is what lets wrappers assume the fast-path memcpy code generation across translation units.

- **Bless representation-stable non-movables:** The trait must bless representation-stable types even when they delete or omit move constructors. Proxy already elides witness move operations when `pro::is_bitwise_trivially_relocatable_v<T>` is `true`; the standard trait needs to make the same guarantee so that non-movable but trivially relocatable witnesses (for example `allocated_ptr`) retain the fast path without faking moves.
- **Keep a declaration-site switch:** Library authors still need a `trivially_relocatable_if_eligible(bool)`-style declaration-site switch. `proxy<F>` is only trivially relocatable when pointer authentication (or other fixups) is disabled and `F::relocatability == constraint_level::trivial`, which echoes the conditional special member rules introduced by [P0848R3]. The syntax lets the implementation describe “trivial if every admitted witness satisfies the trait” without inventing new customization points.

In the shipping implementation, every facade exposes a `relocatability` level that already treats the trivial tier as “bitwise only” via `pro::is_bitwise_trivially_relocatable_v<T>`. Replacing such a predicate with the standard trait would immediately let Proxy drop the bespoke customization point: trivially relocatable witnesses would keep enjoying memcpy-lowered moves (even when non-movable), and other witnesses would continue to fall back to the indirect dispatch. No new user action would be required, and existing facades would stay declarative.

§ 3.5 Lifetime Semantics

Requirement D: Clarify that after a well-formed raw copy of a trivially relocatable object, the destination object lifetime begins immediately and the source lifetime ends, without introducing a new runtime primitive. Investigate tightening `std::launder` or adding targeted wording in [basic.life].

Rationale: Removes the need for a new function name (`restart_lifetime` / `start_lifetime_at`) and leverages existing optimization knowledge.

Open Question Q1: Is a `std::launder` based clarification sufficient for non trivially copyable but trivially relocatable types?

§ 3.6 Guidance (Low Level Usage)

Requirement E: Document that bitwise relocation is safe for common low-level patterns (SBO buffer growth, arena compaction, persistence via `mmap`) when lifetime rules are followed.

Note: If a library chooses to apply pointer authentication or similar signing to wrapper metadata, it must treat the wrapper as non-trivially relocatable regardless of contained value status. No separate “fixup required” trait is proposed; such types simply fail the trivial relocation test.

Open Question Q2: Precise wording path in [basic.life] to end source lifetime on raw copy of a trivially relocatable but non trivially copyable type?

§ 4 Interaction With Existing Papers

§ 4.1 P2786 (Trivial Relocatability For C++26)

[P2786R13] introduced a notion of trivial relocation that explicitly allowed implementation-defined fixups (for example, pointer-authenticated vptr resigning) so that polymorphic types could participate. That scope was inadequate for type erasure: there was no way to detect when the representation was already bitwise-safe, so dispatch-based relocation remained mandatory. National body comments reflected this split. US 46-085 asked that “trivial” mean `memcpy/memmove`, US 47-084 insisted that deleted move operations not preclude trivial relocation, and US 44-082 focused on allowing the annotation to be conditional. The subsequent removal of [P2786R13] from C++26 provides an opportunity to redefine the feature.

§ 4.2 P3780 (Detecting Bitwise Trivially Relocatable Types)

[P3780R0] directly addresses Requirement A and aligns with the NB direction by proposing `is_bitwise_trivially_relocatable`. This proposal is essential if the primary `is_trivially_relocatable` trait retains the broader (fixup-allowing) definition.

§ 4.3 P3858 (`restart_lifetime / start_lifetime_at`)

[P3858R0] introduces a primitive to decompose relocation into copy + lifetime restart. For type erasure fast paths: acceptable if zero overhead and if semantics do not become a generic escape hatch for mutating object representation without constructor/destructor participation. Concerns: multiple invocations, ordering rules, effect on aliasing, and provenance. Requirement D lists constraints. Further committee exploration needed for language wording in [basic.life].

§ 5 Summary

This paper motivates a future trivial relocation design from the perspective of type erasure facilities such as Proxy ([P3086R5]). It clarifies the requirements those libraries need, explains why bitwise relocation unlocks fast paths, and captures production experience as Requirements A-E.

We recommend defining `std::is_trivially_relocatable` strictly around `memcpy / memmove`, owned by the compiler with no user specialization, and keeping non-movable but representation-stable types eligible. Libraries should get a `trivially_relocatable_if_eligible(expr)` switch so fast paths stay conditional on the admitted witnesses.

Lifetime wording should refine the `std::launder / [basic.life]` model, rather than relying on a new relocation-specific `restart_lifetime / start_lifetime_at`-style primitive, while the trait must exclude pointer-auth or other fixup-requiring designs. Publish guidance for SBO growth, arena compaction, and persistence workflows so users apply these rules without falling into UB.

§ 5.1 Next Steps (non-normative)

Draft or contribute wording specifying the trait, conditional syntax details, and lifetime clarifications; coordinate with authors of [P2786R13], [P3780R0], [P3858R0] to unify terminology and ensure removal of prior wording does not regress performance portability.

§ 6 References

[DeSt98] James C. Dehnert and Alexander Stepanov. Fundamentals of Generic Programming.

<https://www.stepanovpapers.com/DeSt98.pdf>

[P0848R3] Barry Revzin and Casey Carter. Conditionally Trivial Special Member Functions.

<https://wg21.link/p0848r3>

[P2786R13] Alisdair Meredith, Mungo Gill, Joshua Berne, Corentin Jabot, Pablo Halpern, and Lori Hughes. Trivial Relocatability For C++26.

<https://wg21.link/p2786r13>

[P3086R5] Mingxin Wang. Proxy: A Pointer-Semantics-Based Polymorphism Library.

<https://wg21.link/p3086r5>

[P3780R0] Giuseppe D’Angelo. Detecting bitwise trivially relocatable types.

<https://wg21.link/p3780r0>

[P3858R0] David Sankel, Jon Bauman, and Pablo Halpern. A Lifetime-Management Primitive for Trivially Relocatable Types.

<https://wg21.link/p3858r0>